AI ASSISTANT CODING ASSIGNMENT -19.2

NAME : MOGILI KEERTHI

ROLL NO : 2303A51427

BATCH NO : 21

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

• Understand how AI tools can assist in translating code between different programming languages. • Learn to verify correctness and functionality after translation.

• Explore syntactic and semantic differences between languages

(e.g., Python, Java, C++).

• Practice debugging and optimizing AI-translated code.

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

• Prompt AI to translate a Python script that reads from and writes to a file into Java.

• Ensure proper handling of file streams, exceptions, and input/output formatting.

• Verify that the Java program produces the same file content and console output as the Python version.

• Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Prompt:

create a python program that can perform file handling operations like read, write, append and delete a file. into a equivalent function in java program and ensure proper handling of streams, exception, and input/output formatting. make sure that the java program should give the same file content and console output as the python version and test both implementation with example inputs file.txt

Code: Python Implementation

```python
# Task-1
"create a python program that can perform file handling operations lik
# Python Implementation
def write_file(filename, content):
    with open(filename, 'w') as file:
        file.write(content)
        print(f"Content written to {filename}")

def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(f"Content of {filename}:\n{content}")
    except FileNotFoundError:
        print(f"{filename} not found.")
def append_file(filename, content):
    with open(filename, 'a') as file:
        file.write(content)
        print(f"Content appended to {filename}")
def delete_file(filename):
    import os
    try:
        os.remove(filename)
        print(f"{filename} has been deleted.")
    except FileNotFoundError:
        print(f"{filename} not found.")
# Example usage
filename = "file.txt"
write_file(filename, "Hello, this is a test file.\n")
read_file(filename)
append_file(filename, "This line is appended to the file.\n")
read_file(filename)
delete_file(filename)
read_file(filename)  # This will show that the file has been deleted.
```

Output:

```
PS C:\AIAC LAB\Assignments Codes> & C:\
thon.exe "c:/AIAC LAB/Assignments Codes
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes>
```

Code: Java Implementation

```java
// Java Implementation
import java.io.*;
class FileHandling {
    public static void writeFile(String filename, String content) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            writer.write(content);
            System.out.println("Content written to " + filename);
        } catch (IOException e) {
            System.err.println("Error writing file: " + e.getMessage());
        }
    }
    public static void readFile(String filename) {
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            System.out.println("Content of " + filename + ":");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (FileNotFoundException e) {
            System.out.println(filename + " not found.");
        } catch (IOException e) {
            System.err.println("Error reading file: " + e.getMessage());
        }
    }
    public static void appendFile(String filename, String content) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename, append: true))) {
            writer.write(content);
            System.out.println("Content appended to " + filename);
        } catch (IOException e) {
            System.err.println("Error appending to file: " + e.getMessage());
        }
    }
    public static void deleteFile(String filename) {
        File file = new File(filename);
        if (file.delete()) {
            System.out.println(filename + " has been deleted.");
        } else {
            System.out.println(filename + " not found.");
        }
    }
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        String filename = "file.txt";
        writeFile(filename, content: "Hello, this is a test file.\n");
        readFile(filename);
        appendFile(filename, content: "This line is appended to the file.\n");
        readFile(filename);
        deleteFile(filename);
        readFile(filename);  // This will show that the file has been deleted.
    }
}
```

Output:

```
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
ming\Code - Insiders\User\workspaceSto
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
dhat.java\jdt_ws\jdt.ls-java-project\b
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes>
```

Observation:

The provided Python and Java implementations perform the same file handling operations: writing to a file, reading from a file, appending to a file, and deleting a file. Both implementations handle exceptions appropriately, ensuring that the user is informed if the file is not found or if an I/O error occurs. The console output for both implementations is consistent, providing clear feedback on the operations performed. The example usage demonstrates the functionality of each operation, ensuring that both implementations yield the same results when tested with the same inputs.

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

• Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).

• Ask AI to generate an equivalent Python implementation.

• Ensure logical correctness and validate results using multiple test cases.

• Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Prompt:

create a C++ program that implements a sorting algorithm into a equivalent function in python program and ensure proper handling of input/output formatting. make sure that the python program should give the same sorted output as the C++ version and test both implementation with example inputs using bubble sort.

Code: C++ Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
    cout << "Original array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
    bubbleSort(arr);
    cout << "Sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

Output: C++ Implementation

```
Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
```

Code: Python Implementation

```python
# Python Implementation
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("\nOriginal array:", arr)
bubble_sort(arr)
print("Sorted array:", arr)
```

Output: Python Implementation

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Observation:

The Python implementation of bubble sort follows the same logic as the C++ version, ensuring the same sorted output. The input and output formatting is consistent between both implementations, allowing for easy testing with the same example inputs. and the output will be the same for both implementations, demonstrating that the sorting algorithm works correctly in both languages.

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

• Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).

• Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.

• Test the generated code using a sample dataset.

• Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Prompt:

create a SQL query that involves group by and aggregate functions into an equivalent function in pandas dataframe and ensure proper handling of data manipulation and output formatting. make sure that the pandas implementation should give the same results as the SQL query and test both implementations with example inputs using a sample dataset.

Code:- SQL Implementation

```sql
-- SQLite

SELECT department, COUNT(*) AS employee_count, AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

Output:- SQL Implementation

```
  department salary
                count      mean
0    Finance     2  62500.0
1         HR     2  52500.0
2         IT     2  85000.0
```

Code:- Python Implementation

```python
# Pandas Implementation
import pandas as pd
# Sample dataset
data = {
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'Finance'],
    'salary': [50000, 80000, 60000, 90000, 55000, 65000]
}
df = pd.DataFrame(data)
# Group by department and calculate aggregate functions
result = df.groupby('department').agg({
    'salary': ['count', 'mean']
}).reset_index()
print(result)
```

Output:- Python Implementation

```
  department salary
                count      mean
0    Finance     2  62500.0
1         HR     2  52500.0
2         IT     2  85000.0
```

Observation:

The provided code snippets demonstrate the implementation of file handling operations in Python and Java, a bubble sort algorithm in C++ and Python, and a SQL query for grouping and aggregating data alongside its equivalent implementation using pandas in Python. Each implementation is designed to produce the same output, ensuring consistency across different programming languages and tools. The examples are tested with sample inputs to validate their correctness and functionality.

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

• Converting a REST API implementation from Python (Flask) to Java (Spring Boot).

• Translating a data processing script from Python to C++ for performance optimization.

Instructions:

• Provide AI with source code in one programming language.

• Prompt AI to convert the code into another chosen language.

• Execute and test both versions to ensure identical functionality.

• Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Prompt:

create a Python script that calculates the standard deviation of a large list of numbers. While Python is great for prototyping, I need to convert this into C++ to optimize for execution speed and memory efficiency. Please provide a functionally equivalent C++ version. Ensure the logic for the mean and variance remains identical, and document the key differences in syntax and memory management between the two.

Code: Python Implementation

```python
# Task-4
# "create a Python script that calculates the standard deviat
# Python Implementation
import math
from numpy import std
from numpy import std
from py_compile import main
def calculate_standard_deviation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    mean = sum(numbers) / n
    variance = sum((x - mean) ** 2 for x in numbers) / n
    return math.sqrt(variance)
# Example usage
numbers = [10, 12, 23, 23, 16, 23, 21, 16]
std_dev = calculate_standard_deviation(numbers)
print(f"\n Standard Deviation: {std_dev}")
```

Output: Python Implementation

```
Standard Deviation: 4.898979485566356
```

Code:

```cpp
# C++ Implementation
#include <iostream>
#include <vector>
#include <cmath>
double calculateStandardDeviation(const std::vector<double>& numbers) {
    int n = numbers.size();
    if (n == 0) {
        return 0;
    }
    double mean = 0;
    for (double num : numbers) {
        mean += num;
    }
    mean /= n;
    double variance = 0;
    for (double num : numbers) {
        variance += (num - mean) * (num - mean);
    }
    variance /= n;
    return std::sqrt(variance);
}
int main() {
    std::vector<double> numbers = {10, 12, 23, 23, 16, 23, 21, 16};
    double stdDev = calculateStandardDeviation(numbers);
    std::cout << "Standard Deviation: " << stdDev << std::endl;
    return 0;
}
```

Output:

```
Standard Deviation: 4.89898
```

Observation:

The above code calculates the standard deviation of a set of numbers using C++. It first computes the mean of the numbers, then calculates the variance by summing the squared differences from the mean, and finally takes the square root of the variance to get the standard deviation. The code is efficient and straightforward, making it easy to understand and maintain. and it demonstrates the use of basic C++ features such as vectors, loops, and functions.

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

• Provide AI with a basic Python function that determines if a number is even or odd.

• Prompt AI to generate equivalent C++ code.

• Ensure correct syntax, input handling, and output formatting.

• Compile and run the C++ program to verify it produces the same result as the Python version.

• Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Prompt:

create a translation of a simple Python function that checks whether a given number is even or odd into an equivalent function in c++. ensure that the logic remains identical and that the output formatting is consistent between both implementations. test both functions with a range of input values to confirm they produce the same results.

Code:

```python
# Task-5
# "create a translation of a simp
# Python Implementation
from ast import main
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
for num in range(1, 11):
    result = check_even_odd(num)
    print(f"{num} is {result}")
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Code:-

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>
std::string checkEvenOdd(int number) {
    if (number % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}
int main() {
    for (int num = 1; num <= 10; num++) {
        std::string result = checkEvenOdd(num);
        std::cout << num << " is " << result << std::endl;
    }
    return 0;
}
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Observation:

The provided code snippet demonstrates a simple Java Spring Boot REST API application. The `SpringBootApiApplication` class serves as the entry point for the application, while the `ApiController` class defines two endpoints: one for a welcome message and another for performing calculations (square and cube) based on a path variable. The use of `Map` allows for flexible responses, and the annotations like `@RestController` and `@GetMapping` facilitate the creation of RESTful endpoints. This structure is typical for a basic Spring Boot application, making it easy to understand and extend in the future.